FIKA, NOT FIKA

Arkitektur och genomförandeplan

En kostnadsmedveten, serverless webbapplikation för bildklassificering på AWS

Serverless	Python	MVP först
Ingen server som står på	Modellen körs nära koden	Enkel att bygga vidare på

Version 1.0 | 29 oktober 2026

Syfte

Dokumentet beskriver en genomförbar första version av applikationen, varför respektive teknik har valts, hur komponenterna samverkar och i vilken ordning lösningen bör byggas. Fokus ligger på låg fast kostnad, tydlig säkerhetsnivå och möjlighet att skala utan att arkitekturen behöver göras om från grunden.

Rekommendation i korthet
Bygg frontend som statiska filer i Amazon S3 och CloudFront. Låt webbläsaren ladda upp bilder direkt till en privat S3-bucket med en tidsbegränsad presigned URL. Kör Python-modellen i AWS Lambda, paketerad som en container image i Amazon ECR. Börja med Lambda Function URL och gå över till API Gateway först när behovet finns.

1. Målbild och avgränsning

Målet är en liten men produktionslik MVP där en extern användare kan välja en bild eller ta ett foto, skicka bilden för analys och få svaret fika, inte fika eller osäkert. Lösningen ska kunna användas publikt utan att AWS-nycklar exponeras och utan en server som debiteras dygnet runt.

Ingår i MVP	Väntar till senare
Uppladdning eller mobilkamera	Användarkonton och social inloggning
Bildvalidering och storleksgräns	Historik per användare
Klassificering med confidence	Avancerad modellövervakning
Automatisk radering av bilder	GPU-baserad inferens
Kostnadslarm och grundläggande loggning	WAF och avancerat missbruksskydd

2. Rekommenderad arkitektur

Arkitekturen separerar användargränssnitt, filöverföring och modellkörning. Det gör att varje del kan ändras oberoende. Modellen kan exempelvis bytas från ONNX Runtime till en annan runtime utan att frontend behöver skrivas om.



Anropsflöde

- Frontend begär en kortlivad presigned URL från backend.

- Webbläsaren laddar upp bilden direkt till S3. Bilddata går alltså inte genom API-lagret.
- Frontend skickar endast S3-nyckeln till klassificeringsfunktionen.
- Lambda kontrollerar objektet, läser bilden och kör modellen.
- Resultatet skickas tillbaka som JSON. Bilden raderas direkt eller av lifecycle-regeln.

3. Teknikval och motivering

Område	Val	Varför
Frontend	React + Vite + TypeScript	Snabb utveckling, liten statisk build och typkontroll. Kräver ingen applikationsserver i produktion. Vanlig JavaScript fungerar också, men TypeScript minskar enkla kontraktsfel mellan frontend och API.
Webbhosting	S3 + CloudFront	Statiska filer är billiga att lagra och leverera. CloudFront ger HTTPS, cache och möjlighet att hålla S3-bucketen privat med Origin Access Control.
API, MVP	Lambda Function URL	Minsta möjliga antal komponenter och ingen separat API Gateway-kostnad. Bra när API:t är litet. CORS begränsas till frontendens origin.
API, senare	API Gateway HTTP API	Läggs till när ni behöver fler routes, JWT, central throttling, versionering eller tydligare API-styrning.
Bildlagring	Privat S3 + presigned PUT	Webbläsaren kan ladda upp direkt utan AWS-nycklar. Lambda slipper hantera stora request bodies och blir enklare och billigare.
Inferens	AWS Lambda	Ingen server står igång. Betalning sker per anrop och körtid. Passar bäst vid låg eller ojämn trafik.
Paketering	Lambda container image + ECR	ML-beroenden är ofta stora. En container gör runtime och native-bibliotek reproducerbara och förenklar lokal testning.
Modellruntime	ONNX Runtime, om modellen stödjer det	Ofta mindre och enklare för inferens än ett komplett träningsramverk. PyTorch eller TensorFlow kan behållas om exporten påverkar kvalitet eller funktion.

Område	Val	Varför
Region	eu-north-1	Rimligt standardval när användarna huvudsakligen finns i Sverige. Bekräfta alltid att önskade tjänster och runtimes finns i regionen.
Infrastructure as Code	AWS SAM	Låg tröskel för en Lambda-dominerad lösning. Ger repeterbar deployment och gör resurser, IAM och konfiguration granskningsbara.

Medvetet bortval i första versionen
Ingen EC2, NAT Gateway, databas eller provisioned concurrency. De komponenterna skapar mer drift, fler felkällor eller löpande kostnad innan vi vet att behovet finns.

4. Backend och modellkontrakt

Klassificeringslogiken bör vara frikopplad från AWS. Kärnan ska kunna testas lokalt med bildbytes och returnera ett stabilt resultatobjekt. Lambda-handlern ansvarar endast för transport, behörighet, validering och felhantering.

Föreslaget resultatkontrakt
classification: "fika" | "not_fika" | "uncertain"
confidence: tal mellan 0 och 1
modelVersion: exempelvis "1.0.0"
requestId: korrelations-ID för felsökning

Modellen laddas utanför handler-funktionen så att en varm Lambda-instans kan återanvända modellen mellan anrop. Det minskar svarstid och CPU-arbete. Börja med 1-2 GB minne och mät, eftersom Lambda ger mer CPU när minnet höjs. Välj sedan den kombination som ger bäst total kostnad och svarstid.

5. Säkerhet, integritet och kostnadsskydd

Filvalidering: Tillåt endast JPEG, PNG och WebP. Kontrollera verkligt filinnehåll, maximal storlek på 5 MB och exempelvis högst 4096 x 4096 pixlar.

Privat lagring: Blockera publik S3-åtkomst. Presigned URL ska vara kortlivad, knuten till en unik UUID-nyckel och helst även till förväntad content type och storlek.

Minsta behörighet: Lambda-rollen får endast läsa och radera objekt under rätt bucket och prefix. Undvik s3:* och generella resurswildcards.

CORS: Tillåt endast den faktiska frontenddomänen och nödvändiga metoder och headers. Undvik wildcard i produktion.

Begränsad kapacitet: Sätt reserved concurrency till exempelvis 2 i början. Det begränsar både kostnad och hur hårt modellen kan belastas.

Dataminimering: Radera bilder efter klassificering när det är möjligt och ha en S3 lifecycle-regel som säkerhetsnät, exempelvis efter ett dygn.

Loggning: Logga request-ID, modellversion, svarstid, status och feltyp. Logga inte själva bilden eller presigned URL:er. Sätt CloudWatch-retention till 7 eller 14 dagar.

Kostnadslarm: Skapa AWS Budgets-notiser vid exempelvis 1, 5 och 10 USD. Budgetar är larm, inte ett omedelbart hårt stopp.

6. Genomförandeplan

Fas	Estimat	Aktiviteter	Klart när
Fas 0 - Definition	0,5 dag	Bestäm vad som räknas som fika, inte fika och osäkert. Dokumentera gränsfall och kvalitetsmål.	Klassdefinition, acceptanskriterier och exempelbilder.
Fas 1 - Modell lokalt	2-5 dagar	Bygg preprocessing, prediction och tester. Versionsmärk modellen och mät precision, recall och confusion matrix på separat testdata.	Ren Python-funktion som tar image bytes och returnerar resultatkontraktet.
Fas 2 - Lokal API-prototyp	1-2 dagar	Lägg ett litet FastAPI-gränssnitt runt modellen. Testa trasiga filer, EXIF-rotation, stora bilder och låg confidence.	Lokalt fungerande API samt automatiska feltester.
Fas 3 - Lambda-container	1-3 dagar	Skapa Dockerfile, bygg för rätt arkitektur och testa med Lambda Runtime Interface Emulator. Mät kallstart, minne och inferenstid.	Container image som kan köras lokalt och i Lambda.
Fas 4 - AWS-grund	1-2 dagar	Skapa ECR, privat upload-bucket, lifecycle, IAM-roll, Lambda, Function URL, CORS, loggretention och budgetlarm via SAM.	Repeterbart dev-miljöpaket i eu-north-1.
Fas 5 - Frontend	2-4 dagar	Bygg uppladdning, kamerastöd, preview, status, felhantering och resultatvy. Lägg frontend i S3 och exponera via CloudFront.	Publikt användbar MVP på HTTPS.
Fas 6 - Hårdning	1-3 dagar	Lägg gränser, concurrency, timeout, larm och negativtest. Kontrollera att bilder raderas och att IAM är	Driftsatt MVP med rimligt missbruks- och kostnadsskydd.

Fas	Estimat	Aktiviteter	Klart när
		minimalt.	
Fas 7 - CI/CD och förbättring	1-2 dagar	Automatisera build, test, ECR push, Lambda update och frontend-deployment.	Kontrollerad releaseprocess och versionsspårning.

Estimatet är avsett för en utvecklare som redan är bekväm med Python och grundläggande webbteknik. Modellträning och datainsamling kan bli den klart största variabeln och bör därför följas separat.

7. Test- och kvalitetsplan

Testnivå	Exempel	Godkänt när
Enhetstest	Preprocessing, filvalidering, label-mapping, confidence-gränser	Reproducerbara resultat och tydliga fel för ogiltiga filer.
Modelltest	Separat testdataset, confusion matrix, bias mot bakgrund eller förpackning	Mål för precision/recall är uppfyllt och kända gränsfall dokumenterade.
Integration	Presigned upload, S3-läsning, klassificering, radering	Hela flödet fungerar utan publika buckets eller exponerade nycklar.
Prestanda	Kall och varm Lambda, olika bildstorlekar	Svarstid och minne ligger inom beslutad budget.
Säkerhet	Fel MIME, polyglot/trasig bild, stor fil, återanvänd object key	Input stoppas tidigt och IAM tillåter bara avsett prefix.
Kostnad	Belastningstest på begränsad nivå, loggvolym, lagringsretention	Budgetlarm fungerar och inga resurser har oväntad fast kostnad.

8. Drift och observability

- Nyckeltal: antal klassificeringar, felandel, p50/p95-svarstid, kallstarter, max memory used och låg-confidence-andel.
- Larm: Lambda errors, throttles, onormalt hög duration och AWS-budget. Börja enkelt och undvik larm som ingen agerar på.
- Modellspårning: returnera och logga modelVersion. Spara inte användarbilden enbart för modellanalys utan ett tydligt beslut och rätt information till användaren.
- Release: publicera ny container-tag, uppdatera Lambda och verifiera ett litet smoke-test. Behåll föregående fungerande image-digest för rollback.

9. Beslutspunkter för nästa nivå

Signal	Åtgärd
Fler routes, JWT eller central throttling behövs	Inför API Gateway HTTP API.
Publik endpoint missbrukas	Inför CAPTCHA och därefter WAF/rate limiting där kostnad och behov motiverar det.
Kallstart påverkar användarupplevelsen	Optimera image/modell först. Överväg provisioned concurrency först efter mätning.
Modellen kräver GPU eller är för stor	Flytta inferens till SageMaker, ECS/Fargate eller annan lämplig modellserver.
Stadig hög trafik dygnet runt	Jämför Lambda-kostnad med container som hålls varm.
Historik eller feedback behövs	Lägg till DynamoDB först när datamodell och retention är tydliga.

10. Rekommenderad första sprint

Sprintmål
En användare ska kunna öppna en HTTPS-sida, välja en bild, få ett klassificeringsresultat och vara trygg med att bilden inte ligger kvar permanent.

- Dag 1: klassdefinition, modellkontrakt och repo-struktur.
- Dag 2-3: lokal modellfunktion, tester och baseline-mätning.
- Dag 4: Lambda-container och lokal invocation.
- Dag 5: SAM-stack med S3, IAM, Lambda, Function URL och budget.
- Dag 6-7: React-frontend, presigned upload och resultatvy.
- Dag 8: säkerhetstest, kostnadskontroll, dokumentation och release.

11. Källor och antaganden

Arkitekturvalen bygger på AWS officiella dokumentation. Priser och Free Tier-villkor kan ändras och bör kontrolleras i det aktuella AWS-kontot innan produktion.

- AWS Lambda Pricing - free tier, begäranden och GB-sekunder: https://aws.amazon.com/lambda/pricing/
- AWS API Gateway Pricing - HTTP API och free tier: https://aws.amazon.com/api-gateway/pricing/
- AWS Lambda - Deploy Python functions with container images: https://docs.aws.amazon.com/lambda/latest/dg/python-image.html
- Amazon S3 - Uploading objects with presigned URLs: https://docs.aws.amazon.com/AmazonS3/latest/userguide/PresignedUrlUploadObject.html
- Amazon CloudFront - Secure static website with S3 and OAC: https://docs.aws.amazon.com/AmazonCloudFront/latest/DeveloperGuide/getting-started-secure-static-website-cloudformation-template.html
- AWS Budgets - Managing costs and alerts: https://docs.aws.amazon.com/cost-management/latest/userguide/budgets-managing-costs.html